# Reporte Técnico de Actividades Práctico-Experimentales Nro. 005

## 1. Datos de Identificación del Estudiante y la Práctica

| | |
|---|---|
| **Nombre del estudiante(s)** | Austin Erreyes |
| **Asignatura** | Teoría de la programación |
| **Ciclo** | 1 A |
| **Unidad** | 1 |
| **Resultado de aprendizaje de la unidad** | Identifica los conceptos fundamentales de la teoría de la programación, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad. |
| **Práctica Nro.** | 005 |
| **Título de la Práctica** | Instalación de lenguajes de programación. |
| **Nombre del Docente** | Jose Oswaldo Guaman Quinche |
| **Fecha** | Jueves 22 de octubre del 2026 al<br>Viernes 23 de octubre del 2026 |
| **Horario** | Jueves, 12h30 – 13h30<br>viernes, 09h30 - 11h30 |
| **Lugar** | Aula física asignada al paralelo. |
| **Tiempo planificado en el Sílabo** | 3 horas |

## 2. Objetivo(s) de la Práctica

- Desarrollar la capacidad de transformar un problema en una solución computacional.
- Aplicar estructuras secuenciales en el diseño del algoritmo en un lenguaje de programación.
- Validar la lógica del algoritmo mediante pruebas de escritorio

## 3. Materiales, Reactivos, Equipos y Herramientas

a. IDE de programación: Visual Studio Code u otro entorno compatible.
b. Lenguaje de programación: Según los contenidos de la unidad.

c. Computador personal con sistema operativo Windows, Linux o macOS.
d. Material de apoyo en el Entorno Virtual de Aprendizaje (EVA).

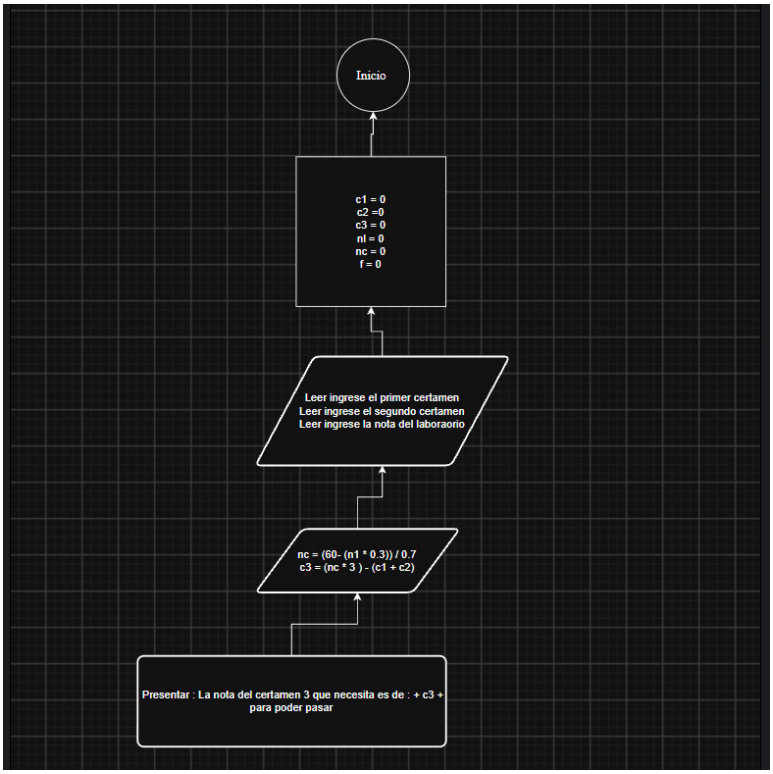
e. Editores de texto (Word, Google Docs u otros) para la elaboración del informe técnico en formato PDF.
f. Conexión a internet estable para acceder a recursos digitales y software en línea.

## 4. Procedimiento / Metodología Ejecutada

## 5. Resultados

Prueba de escritorio

Diagrama de flujo

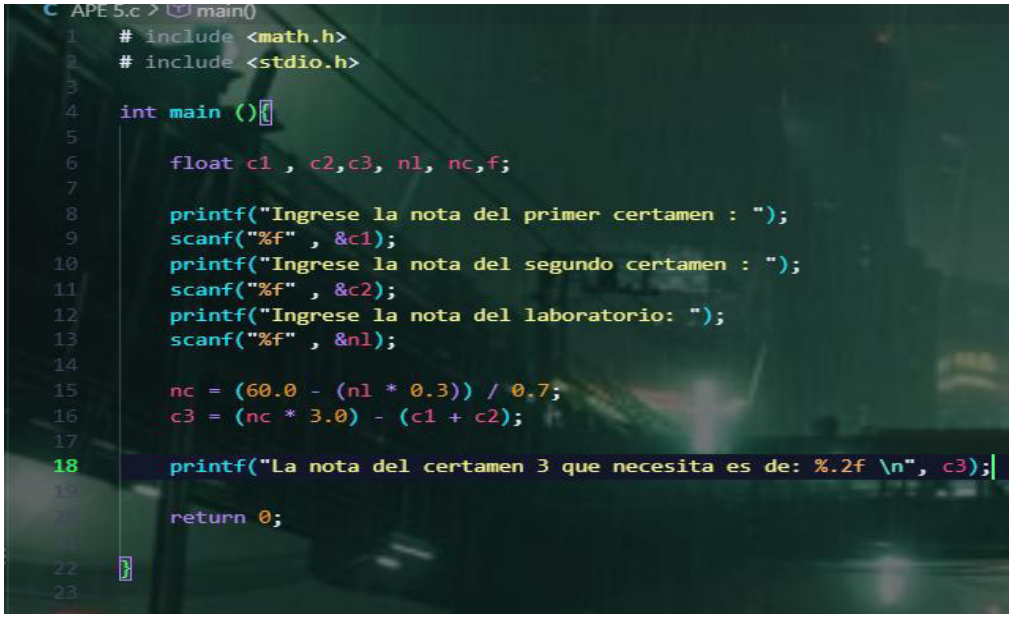



Seudocodigo

```
Algoritmo CalculoCertamen3
   Definir c1, c2, c3, nl, nc Como Real
   c1 <- 0.0
   c2 <- 0.0
   c3 <- 0.0
   nl <- 0.0
   nc <- 0.0
      Escribir "Ingrese la nota del primer certamen: "
   Leer c1
   Escribir "Ingrese la nota del segundo certamen: "
   Leer c2
```

```
Escribir "Ingrese la nota del laboratorio: "
Leer nl
nc <- (60.0 - (nl * 0.3)) / 0.7
c3 <- (nc * 3.0) - (c1 + c2)
Escribir "---------- RESULTADOS DEL ANALISIS ----------"
Escribir "La nota del certamen 3 que necesita es de: ", c3
Si (c3 > 100.0) Entonces
   Escribir "Estado: Meta inalcanzable (necesita mas de 100 puntos)."
SiNo
   Si (c3 <= 0.0) Entonces
      Escribir "Estado: ¡Felicidades! Ya aprobo la asignatura (necesita 0 o menos puntos)."
   SiNo
      Escribir "Estado: Meta alcanzable. Debe esforzarse para obtener esa nota en el certamen 3."
   FinSi
FinSi
Escribir "----------------------------------------------------------------"
Escribir "[INFO]: Proceso completado exitosamente en el sistema academico."
FinAlgoritmo
```

Codigo



```c
# include <math.h>
# include <stdio.h>

int main (){

    float c1 , c2,c3, nl, nc,f;

    printf("Ingrese la nota del primer certamen : ");
    scanf("%f" , &c1);
    printf("Ingrese la nota del segundo certamen : ");
    scanf("%f" , &c2);
    printf("Ingrese la nota del laboratorio: ");
    scanf("%f" , &nl);

    nc = (60.0 - (nl * 0.3)) / 0.7;
    c3 = (nc * 3.0) - (c1 + c2);

    printf("La nota del certamen 3 que necesita es de: %.2f \n", c3);

    return 0;

}
```

## 6. Preguntas de Control

**¿Qué es un programa con estructuras secuenciales?**

Es aquel en el que las instrucciones se ejecutan de manera sucesiva, una detrás de otra, siguiendo un orden lineal de principio a fin. En el documento adjunto, esto se evidencia claramente en el algoritmo CalculoCertamen3 y su código en C, donde el flujo avanza por etapas ordenadas: primero se solicitan y leen las variables de entrada (c1, c2, nl), a continuación, se ejecutan las fórmulas

matemáticas secuenciales para calcular nc y c3, y por último se muestran los resultados en pantalla.

**¿Cómo se traslada un algoritmo en pseudocódigo o diagrama de flujo a un lenguaje de programación?**

Se logra transformando y adaptando cada instrucción del diseño previo a la sintaxis formal del lenguaje seleccionado.

**¿Por qué es importante validar el programa mediante casos de prueba?**

Es esencial para comprobar que la lógica implementada tanto en el diagrama de flujo y el pseudocódigo como en el código fuente produce los resultados correctos y esperados. Permite verificar que las operaciones matemáticas funcionen adecuadamente ante distintos escenarios de entrada, asegurando que el programa esté libre de errores de lógica o de sintaxis antes de su ejecución definitiva.

## 7. Conclusiones

- Permite transformar problemas reales en soluciones computacionales funcionales mediante el diseño estructurado de algoritmos.
- Garantiza la ejecución ordenada y sucesiva de instrucciones al pasar del diagrama de flujo y el pseudocódigo al lenguaje de programación.
- Valida la lógica algorítmica y la precisión matemática mediante pruebas de escritorio antes de escribir el código fuente.

## 8. Recomendaciones

No hay recomendaciones en este APE.

## 9. Bibliografía / Referencias

[1] A. A. Bhuiyan and M. Amiruzzaman, Programming with Java, 2nd ed. The Pennsylvania Alliance for Design of Open Textbooks (PA-ADOPT), 2025. Disponible en: https://open.umn.edu/opentextbooks/textbooks/programmingwith-java

[2] J. E. Guerra Salazar, M. V. Ramos Valencia, and G. E. Vallejo Vallejo, Programando en C desde la práctica: problemas resueltos. Puerto Madero Editorial, 2023. Disponible en: https://dialnet.unirioja.es/servlet/libro?codigo=933288

[3] J. M. Toro Bonilla, Fundamentos de programación: Python, 2.ª ed. Editorial Universidad de Sevilla, 2023. Disponible en: https://dialnet.unirioja.es/servlet/libro?codigo=871117

## 10. Anexos

https://github.com/Aust1n-Drgon/Sistemas.git